

Celebal Excellence Internship

[Week – 3 Task – Subqueries, CTE, Window Functions]

Title: CEI – week3 task submission

Name: Anushka Manoj Patil

College name: Dr. D. Y. Patil Pimpri, Pune

Step 1: Setup Data

1. Table creation:

- Creating first table CUSTOMERS

Query:

```
CREATE TABLE CUSTOMERS
(
  CUSTOMER_ID VARCHAR(20) PRIMARY KEY,
  CUSTOMER_NAME VARCHAR(100),
  SEGMENT VARCHAR(30)
);
```

- Creating second table PRODUCTS

Query:

```
CREATE TABLE PRODUCTS
(
  PRODUCT_ID VARCHAR(30),
  PRODUCT_NAME VARCHAR(200),
  CATEGORY VARCHAR(50),
  SUB_CATEGORY VARCHAR(50)
);
```

- Creating third table : ORDERS

Query:

```
CREATE TABLE ORDERS
(
  ROW_ID INT,
  ORDER_ID VARCHAR(30),
```

```
ORDER_DATE DATE,  
SHIP_DATE DATE,  
SHIP_MODE VARCHAR(50),  
CUSTOMER_ID VARCHAR(20),  
COUNTRY VARCHAR(50),  
CITY VARCHAR(50),  
STATE_NAME VARCHAR(50),  
POSTAL_CODE INT,  
REGION VARCHAR(30),  
PRODUCT_ID VARCHAR(30),  
SALES DECIMAL(10,2),  
QUANTITY INT,  
DISCOUNT DECIMAL(5,2),  
PROFIT DECIMAL(10,2)  
);
```

2. Inserting values in table

- Inserting values into CUSTOMERS table,
by extracting from superstore dataset

Query:

```
INSERT INTO CUSTOMERS  
SELECT DISTINCT  
[CUSTOMER_ID],  
[CUSTOMER_NAME],  
SEGMENT  
FROM SUPERSTORE_RAW;
```

- Inserting values into PRODUCTS table,
by extracting from superstore dataset

Query:

```
INSERT INTO Products
SELECT DISTINCT
    [Product_ID],
    [Product_Name],
    Category,
    [Sub_Category]
FROM superstore_raw;
```

- Inserting values into ORDERS table,
by extracting from superstore dataset

Query:

```
INSERT INTO Orders
(
    ROW_ID,
    ORDER_ID,
    ORDER_DATE,
    SHIP_DATE,
    SHIP_MODE,
    CUSTOMER_ID,
    COUNTRY,
    CITY,
    STATE_NAME,
    POSTAL_CODE,
    REGION,
    PRODUCT_ID,
    SALES,
    QUANTITY,
    DISCOUNT,
    PROFIT
)
SELECT
    ROW_ID,
    ORDER_ID,
    TRY_CONVERT(DATE, ORDER_DATE, 101),
```

```
TRY_CONVERT(DATE, SHIP_DATE, 101),  
SHIP_MODE,  
CUSTOMER_ID,  
COUNTRY,  
CITY,  
STATE,  
POSTAL_CODE,  
REGION,  
PRODUCT_ID,  
SALES,  
QUANTITY,  
DISCOUNT,  
PROFIT  
FROM SUPERSTORE_RAW;
```

Step 2: Perform Required Queries

Q1. Find all orders where sales are greater than the average sales.
(Subquery)

Query:

```
SELECT *  
FROM ORDERS  
WHERE SALES >  
(  
    SELECT AVG(SALES)  
    FROM ORDERS  
);
```

Result:

	ROW_ID	ORDER_ID	ORDER_DATE	SHIP_DATE	SHIP_MODE	CUSTOMER_ID	COUNTRY	CITY
1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	United States	Henderson
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	United States	Henderson
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	United States	Fort Lauderdale
4	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	United States	Los Angeles
5	11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	United States	Los Angeles
6	12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	United States	Los Angeles
7	14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	United States	Seattle
8	17	CA-2014-105893	2014-11-11	2014-11-18	Standard Class	PK-19075	United States	Madison

CITY	STATE_NAME	POSTAL_CODE	REGION	PRODUCT_ID	SALES	QUANTITY	DISCOUNT	PROFIT
Henderson	Kentucky	42420	South	FUR-BO-10001798	261.96	2	0.00	41.91
Henderson	Kentucky	42420	South	FUR-CH-10000454	731.94	3	0.00	219.58
Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	957.58	5	0.45	-383.03
Los Angeles	California	90032	West	TEC-PH-10002275	907.15	6	0.20	90.72
Los Angeles	California	90032	West	FUR-TA-10001539	1706.18	9	0.20	85.31
Los Angeles	California	90032	West	TEC-PH-10002033	911.42	4	0.20	68.36
Seattle	Washington	98103	West	OFF-BI-10003656	407.98	3	0.20	132.59
Madison	Wisconsin	53711	Central	OFF-ST-10004186	665.88	6	0.00	13.32

Observation:

This query returns the orders in which sales are greater than the average sales by using the subquery,

Here subquery will execute first and result of the subquery will get compared with where clause condition. And final output will be evaluated.

Q2. Find the highest sales order for each customer. (Subquery)

Query:

```
SELECT
ORDER_ID,
CUSTOMER_ID,
SALES,
ORDER_DATE
FROM ORDERS O
WHERE SALES =
```

```
(
  SELECT MAX(SALES)
  FROM ORDERS
  WHERE CUSTOMER_ID = O.CUSTOMER_ID
)
ORDER BY CUSTOMER_ID;
```

Result:

ORDER_ID	CUSTOMER_ID	SALES	ORDER_DATE
CA-2015-110863	AA-10645	1323.90	2015-11-17
CA-2016-140935	AB-10015	341.96	2016-11-10
CA-2016-129714	AB-10060	4355.17	2016-09-01
CA-2016-117121	AB-10105	9892.74	2016-12-17
CA-2014-169061	AB-10150	479.97	2014-03-05
CA-2015-115945	AB-10165	304.23	2015-06-25
CA-2017-126662	AB-10255	479.98	2017-07-17
US-2017-155425	AB-10600	899.14	2017-11-10

Observation:

This query finds highest sales order for each customer using the subquery.

Q3. Calculate total sales for each customer.(CTE)

Query:

```
WITH CustomerSales AS
(
  SELECT
    CUSTOMER_ID,
    SUM(SALES) AS TotalSales
  FROM ORDERS
```

```

GROUP BY CUSTOMER_ID
)
SELECT *
FROM CustomerSales
ORDER BY TotalSales DESC;

```

Result:

CUSTOMER_ID	TotalSales
TA-21385	14595.62
AB-10105	14473.57
KL-16645	14175.23
SC-20095	14142.34
HL-15040	12873.30
SE-20110	12209.44
CC-12370	12129.08
TS-21370	11891.75

Observation:

This query first calculates total sales by using the CTE i.e. Common Table Expression.

Then arranges it in descending order by using the ORDER BY clause.

Q4. Find customers whose total sales are above average. (CTE + Subquery)

Query:

```

WITH CustomerSales AS
(
  SELECT
    CUSTOMER_ID,
    SUM(SALES) AS TotalSales
  FROM ORDERS
  GROUP BY CUSTOMER_ID
)

```

```

SELECT *
FROM CustomerSales
WHERE TotalSales >
(
    SELECT AVG(TotalSales)
    FROM CustomerSales
);

```

Result:

CUSTOMER_ID	TotalSales
LF-17185	3930.52
KH-16330	3312.87
SC-20305	3979.06
SV-20785	4105.32
JL-15505	3173.87
RB-19360	15117.35
FP-14320	4046.75
HR-14830	5248.80

Observation:

This query first calculates total sales by using the CTE i.e Common Table Expression.

It then calculates the average sales by using the aggregate function i.e. AVG().

Then compares total sales with average sales.

Q5. Rank all customers based on total sales. (Window Function)

Query:

```

WITH CustomerSales AS
(
    SELECT
        CUSTOMER_ID,
        SUM(SALES) AS TotalSales

```

```

FROM ORDERS
GROUP BY CUSTOMER_ID
)
SELECT
CUSTOMER_ID,
TotalSales,
RANK() OVER
(
ORDER BY TotalSales DESC
) AS SalesRank
FROM CustomerSales
ORDER BY SalesRank;

```

Result:

	CUSTOMER_ID	TotalSales	SalesRank
1	SM-20320	25043.07	1
2	TC-20980	19052.22	2
3	RB-19360	15117.35	3
4	TA-21385	14595.62	4
5	AB-10105	14473.57	5
6	KL-16645	14175.23	6
7	SC-20095	14142.34	7
8	HL-15040	12873.30	8

	CUSTOMER_ID	TotalSales	SalesRank
21	JL-15835	9799.94	21
22	GT-14635	9351.20	22
23	HW-14935	9300.25	23
24	TB-21400	9133.99	24
25	PF-19120	9062.86	25
26	CM-12385	8954.01	26
27	JD-16150	8828.04	27
28	JE-15715	8697.86	28

Observation:

This query first calculates total sales by using the CTE i.e Common Table Expression.

It then ranks the sales by using the window function i.e.RANK()

Then uses ORDER BY clause to arrange it in descending order.

Q6. Assign row numbers to each order within a customer. (Window Function + PARTITION BY)

Query:

```
SELECT
ORDER_ID,
CUSTOMER_ID,
SALES,
ORDER_DATE,
ROW_NUMBER() OVER
(
PARTITION BY CUSTOMER_ID
ORDER BY ORDER_DATE
) AS RowNumber
FROM ORDERS
ORDER BY CUSTOMER_ID, RowNumber;
```

Result:

	ORDER_ID	CUSTOMER_ID	SALES	ORDER_DATE	RowNumber
1	CA-2014-128055	AA-10315	673.57	2014-03-31	1
2	CA-2014-128055	AA-10315	52.98	2014-03-31	2
3	CA-2014-138100	AA-10315	14.94	2014-09-15	3
4	CA-2014-138100	AA-10315	14.56	2014-09-15	4
5	CA-2015-121391	AA-10315	26.96	2015-10-04	5
6	CA-2016-103982	AA-10315	3930...	2016-03-03	6
7	CA-2016-103982	AA-10315	2.30	2016-03-03	7
8	CA-2016-103982	AA-10315	431.98	2016-03-03	8

	ORDER_ID	CUSTOMER_ID	SALES	ORDER_DATE	RowNumber
337	CA-2015-130113	AH-10030	668.16	2015-12-27	9
338	CA-2016-162747	AH-10030	86.45	2016-03-20	10
339	CA-2017-164000	AH-10030	18.70	2017-12-18	11
340	CA-2014-160066	AH-10075	5.22	2014-11-16	1
341	CA-2015-161795	AH-10075	3.01	2015-12-21	2
342	CA-2016-144015	AH-10075	231.72	2016-06-26	3
343	CA-2016-144015	AH-10075	17.90	2016-06-26	4
344	CA-2016-144015	AH-10075	12.48	2016-06-26	5

Observation:

This query assigns number to each row by using the window function i.e. ROW_NUMBER().

Also partitions it by CUSTOMER_ID.

Q7. Display top 3 customers based on total sales. (Window Function)

Query:

```
WITH CustomerSales AS
(
  SELECT
    CUSTOMER_ID,
```

```

SUM(SALES) AS TotalSales
FROM ORDERS
GROUP BY CUSTOMER_ID
)
SELECT
CUSTOMER_ID,
TotalSales
FROM
(
SELECT
CUSTOMER_ID,
TotalSales,
RANK() OVER
(
ORDER BY TotalSales DESC
) AS SalesRank
FROM CustomerSales
) AS RankedCustomers
WHERE SalesRank < = 3
ORDER BY SalesRank;

```

Result:

	CUSTOMER_ID	TotalSales
1	SM-20320	25043.07
2	TC-20980	19052.22
3	RB-19360	15117.35

Observation:

This query first calculates total sales by using the CTE i.e Common Table Expression.

It then ranks all rows by using RANK() window function.

Then filters and displays the only top 3 rows.

Step 3: Final Combined Query

Q. Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

(Use **JOIN + CTE + Window Function together**)

Query:

```
WITH CustomerSales AS
(
    SELECT
        C.CUSTOMER_ID,
        C.CUSTOMER_NAME,
        SUM(O.SALES) AS TotalSales
    FROM CUSTOMERS C
    JOIN ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
    GROUP BY
        C.CUSTOMER_ID,
        C.CUSTOMER_NAME
)
```

```

SELECT
CUSTOMER_NAME,
TotalSales,
RANK() OVER
(
ORDER BY TotalSales DESC
) AS SalesRank
FROM CustomerSales
ORDER BY SalesRank;

```

Result:

	CUSTOMER_NAME	TotalSales	SalesRank
1	Sean Miller	25043.07	1
2	Tamara Chand	19052.22	2
3	Raymond Buch	15117.35	3
4	Tom Ashbrook	14595.62	4
5	Adrian Barton	14473.57	5
6	Ken Lonsdale	14175.23	6
7	Sanjit Chand	14142.34	7
8	Hunter Lopez	12873.30	8
9	Sanjit Engle	12209.44	9
10	Christopher Conant	12129.08	10
11	Todd Sumrall	11891.75	11
12	Greg Tran	11820.12	12
13	Becky Martin	11789.64	13
14	Seth Vernon	11470.94	14
15	Caroline Jumper	11164.97	15
16	Clay Ludtke	10880.56	16
17	Maria Etezadi	10663.73	17
18	Karen Ferguson	10604.26	18
19	Bill Shonely	10501.65	19
20	Edward Hooks	10310.87	20
21	John Lee	9799.94	21
22	Grant Thornton	9351.20	22
23	Helen Wasserman	9300.25	23
24	Tom Boeckenhauer	9133.99	24
25	Peter Fuller	9062.86	25
26	Christopher Martin...	8954.01	26
27	Justin Deggeller	8828.04	27
28	Joe Elijah	8697.86	28
29	Laura Armstrong	8673.23	29
30	Pete Kriz	8646.93	30
31	Daniel Raglin	8350.87	31
32	Natalie Fritzler	8322.82	32
33	Karen Daniels	8282.36	33
34	Nick Crebassa	8241.74	34

Observation:

This query first calculates total sales by using CTE and uses Inner join to join two tables.

Also uses window function i.e. RANK() to assign unique number to each row.

Mini Project: Customer Sales Insights

Q1. Who are the top 5 customers?

Query:

```
WITH CustomerSales AS
(
    SELECT
        C.CUSTOMER_ID,
        C.CUSTOMER_NAME,
        SUM(O.SALES) AS TotalSales
    FROM CUSTOMERS C
    JOIN ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
    GROUP BY
        C.CUSTOMER_ID,
        C.CUSTOMER_NAME
)
SELECT
    CUSTOMER_ID,
    CUSTOMER_NAME,
    TotalSales,
    SalesRank
```

```

FROM
(
  SELECT *,
  RANK() OVER
  (
    ORDER BY TotalSales DESC
  )AS SalesRank
  FROM CustomerSales
) AS RankedCustomers
WHERE SalesRank <= 5
ORDER BY SalesRank;

```

Result:

	CUSTOMER_ID	CUSTOMER_NAME	TotalSales	SalesRank
1	SM-20320	Sean Miller	25043.07	1
2	TC-20980	Tamara Chand	19052.22	2
3	RB-19360	Raymond Buch	15117.35	3
4	TA-21385	Tom Ashbrook	14595.62	4
5	AB-10105	Adrian Barton	14473.57	5

Observation:

For displaying top 5 customers the query first calculates total sales by using CTE through joining to tables.

Then it ranks the sales by using the window function RANK().

Then filters the rows by WHERE clause.

Q2. Who are the Bottom 5 Customers?

Query:

```

WITH CustomerSales AS
(

```

```
SELECT
C.CUSTOMER_ID,
C.CUSTOMER_NAME,
SUM(O.SALES) AS TotalSales
FROM CUSTOMERS C
JOIN ORDERS O
ON C.CUSTOMER_ID = O.CUSTOMER_ID
GROUP BY
C.CUSTOMER_ID,
C.CUSTOMER_NAME
)
SELECT
CUSTOMER_ID,
CUSTOMER_NAME,
TotalSales,
SalesRank
FROM
(
  SELECT *,
  RANK() OVER
  (
    ORDER BY TotalSales ASC
  ) AS SalesRank
  FROM CustomerSales
) AS RankedCustomers
WHERE SalesRank <= 5
ORDER BY SalesRank;
```

Result:

	CUSTOMER_ID	CUSTOMER_NAME	TotalSales	SalesRank
1	TS-21085	Thais Sissman	4.84	1
2	LD-16855	Lela Donovan	5.30	2
3	CJ-11875	Carl Jackson	16.52	3
4	MG-18205	Mitch Gastineau	16.74	4
5	RS-19870	Roy Skaria	22.33	5

Observation:

For displaying top 5 customers the query first calculates total sales by using CTE through joining to tables.

Then it ranks the sales by using the window function RANK().

Then filters the rows by WHERE clause.

Q3. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Query:

```
WITH CustomerOrders AS
(
    SELECT
        CUSTOMER_ID,
        COUNT(ORDER_ID) AS Totalorders
    FROM ORDERS
    GROUP BY CUSTOMER_ID
    HAVING COUNT(ORDER_ID) = 1
)
SELECT
    C.CUSTOMER_ID,
    C.CUSTOMER_NAME,
    CO.TotalOrders
```

```
FROM CUSTOMERS C
JOIN CustomerOrders CO
ON C.CUSTOMER_ID = CO.CUSTOMER_ID;
```

Result:

	CUSTOMER_ID	CUSTOMER_NAME	TotalOrders
1	CJ-11875	Carl Jackson	1
2	RE-19405	Ricardo Emerson	1
3	JR-15700	Jocasta Rupert	1
4	AO-10810	Anthony O'Donnell	1
5	LD-16855	Lela Donovan	1

Observation:

This query first calculates Total count of orders by using aggregate function COUNT() inside the CTE.

Then by using HAVING it groups the orders with having count 1.

Q4. Which Customers Have Above-Average Sales?

Query:

```
WITH CustomerSales AS
(
  SELECT
    CUSTOMER_ID,
    SUM(SALES) AS TotalSales
  FROM ORDERS
  GROUP BY CUSTOMER_ID
)
SELECT *
FROM CustomerSales
WHERE TotalSales >
```

```
(  
    SELECT AVG(TotalSales)  
    FROM CustomerSales  
);
```

Result:

	CUSTOMER_ID	TotalSales
4	SV-20785	4105.32
5	JL-15505	3173.87
6	RB-19360	15117.35
7	FP-14320	4046.75
8	HR-14830	5248.80
9	JM-15250	3159.11
10	SR-20740	4345.88
11	DG-13300	3195.82

Observation:

This query first calculates sum of orders by using aggregate function SUM() inside the CTE.

Then by using the subquery it calculates average of total sales.

And compares total sales with average sales.

Q5. What is the Highest Order Value per Customer?

Query:

```
WITH RankedOrders AS  
(  
    SELECT  
        O.ORDER_ID,  
        C.CUSTOMER_ID,  
        C.CUSTOMER_NAME,  
        O.SALES,  
        ROW_NUMBER() OVER
```

```

(
  PARTITION BY C.CUSTOMER_ID
  ORDER BY O.SALES DESC
) AS RowNum
FROM ORDERS O
JOIN CUSTOMERS C
ON O.CUSTOMER_ID = C.CUSTOMER_ID
)
SELECT
CUSTOMER_ID,
CUSTOMER_NAME,
ORDER_ID,
SALES AS HighestorderValue
FROM RankedOrders
WHERE RowNum = 1
ORDER BY HighestOrderValue DESC;

```

Result:

	CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	HighestorderValue
1	SM-20320	Sean Miller	CA-2014-145317	22638.48
2	TC-20980	Tamara Chand	CA-2016-118689	17499.95
3	RB-19360	Raymond Buch	CA-2017-140151	13999.96
4	TA-21385	Tom Ashbrook	CA-2017-127180	11199.97
5	HL-15040	Hunter Lopez	CA-2017-166709	10499.97
6	AB-10105	Adrian Barton	CA-2016-117121	9892.74
7	SC-20095	Sanjit Chand	CA-2014-116904	9449.95
8	BS-11365	Bill Shonely	US-2016-1074...	9099.93

Observation:

This query uses the window function inside CTE to give unique rank number to each row and joins two tables.

Then it filters by using the where clause and displays the rows which having row number as 1.